

Corso di Laurea in Ingegneria e Scienze Informatiche

Un linguaggio per domarli tutti: applicazioni full stack con librerie native in Kotlin Multiplatform

Tesi di laurea in:
PROGRAMMAZIONE AD OGGETTI

Relatore

Prof. Danilo Pianini

Candidato

Ouakani Sohaib

Correlatori

Ing. Venusti Valter

Dott. Cortecchia Angela

Abstract

In contesti industriali legati a ingegneria e simulazione, lo stack software è spesso eterogeneo: le librerie che implementano standard tecnici sono scritte in C o C++, mentre i componenti applicativi di alto livello usano linguaggi diversi, causando duplicazione di codice e costi di manutenzione crescenti.

Questa tesi affronta tale problema progettando e sviluppando un applicativo full stack realizzato interamente in Kotlin per la gestione e la simulazione di modelli nel formato FMU (Functional Mock-up Unit), definito dallo standard FMI (Functional Mock-up Interface) per l'interscambio di modelli di simulazione, integrando la libreria nativa FMILib scritta in C.

Il sistema adotta un'architettura client-server modulare: un nucleo di logica di dominio per il ciclo di vita degli FMU, un backend che espone le funzionalità tramite API REST, e un frontend Compose Multiplatform per configurare simulazioni e visualizzarne i risultati. L'interoperabilità con FMILib è realizzata tramite cinterop di Kotlin/Native, mentre Kotlin Multiplatform condivide la logica applicativa tra Linux x64, macOS ARM64 e Windows x64, producendo eseguibili nativi autonomi. Per macOS ARM64 è stata sviluppata una pipeline che ricompila gli FMU privi di binari per Apple Silicon a partire dai sorgenti C distribuiti.

Il sistema è stato validato tramite test unitari e di integrazione eseguiti su tutte le piattaforme target in una pipeline di integrazione continua, confermando la correttezza funzionale e la portabilità del sistema.

Il lavoro dimostra la fattibilità di un approccio monolinguaggio per applicativi che interagiscono con librerie native, evidenziando anche i limiti attuali dell'ecosistema Kotlin/Native che ne condizionano la maturità per un'adozione industriale più ampia.

Opzionale. Poche righe al massimo.

Contents

Abstract	iii
1 Introduzione	1
1.1 Contesto e Obiettivi del progetto	1
1.2 Background	2
1.2.1 Sviluppo di applicazioni full stack monolinguaggio	2
1.2.2 Kotlin e framework multitarget come risposta al gap	5
1.2.3 Interoperabilità con C tramite Cinterop	7
1.2.4 Lo standard FMI e il formato FMU	8
2 Analisi	13
2.1 Requisiti	13
2.2 Modello del dominio	14
2.2.1 Vincoli	16
3 Design	17
3.1 Architettura	17
3.2 Design di dettaglio	19
3.2.1 Interfacciamento con librerie native	19
3.2.2 Backend multiplatforma	21
3.2.3 Frontend multiplatforma	22
4 Implementazione	25
4.1 Binding C/Kotlin e configurazione del linker per piattaforma	26
4.2 Gestione del ciclo di vita dell'FMU	27
4.3 Estrazione dei metadati e gestione delle variabili	28
4.4 Esecuzione della simulazione	29
4.5 Ricompilazione degli FMU su macOS	
ARM64	31
4.6 Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma	33
4.7 Implementazione backend	36

CONTENTS

4.8	Implementazione frontend	37
5	Valutazione	39
5.1	Test unitari e di integrazione	39
5.2	Copertura multiplatforma nella CI	41
6	Conclusioni e Sviluppi Futuri	43
6.1	Limitazioni	43
6.2	Sviluppi futuri	44
6.3	Conclusioni	45
		47
	Bibliography	47

Chapter 1

Introduzione

1.1 Contesto e Obiettivi del progetto

In molti contesti industriali, lo stack software degli applicativi interni è eterogeneo per necessità: le librerie che implementano standard tecnici di settore sono scritte in linguaggi di programmazione nativi, mentre i componenti di alto livello – interfacce utente, logica applicativa, servizi di comunicazione – sono sviluppati in linguaggi diversi. Questa stratificazione porta a duplicazione di codice, aumento della complessità strutturale e costi di manutenzione crescenti.

L'esigenza di unificare le tecnologie impiegate, adottando un unico linguaggio per l'intero stack senza rinunciare all'integrazione con le librerie native esistenti, è riscontrata da diverse aziende operanti in ambito ingegneristico e di simulazione come Dallara, rendendo il problema da cui prende origine questo lavoro di rilevanza non solo teorica, ma anche pratica. Una possibile risposta a questa esigenza è rappresentata da Kotlin Multiplatform, che permette di condividere logica applicativa tra piattaforme diverse producendo binari nativi tramite il compilatore Kotlin/Native, e che offre un meccanismo diretto per interfacciarsi con librerie C tramite lo strumento `cinterop`, aspetti che verranno in seguito approfonditi nel Capitolo 1.2.

L'obiettivo del progetto è la progettazione e lo sviluppo di un applicativo full stack in Kotlin per la gestione e la simulazione di file Functional Mock-up Unit (FMU), il formato archivistico definito dallo standard Functional Mock-up

Interface (FMI) per lo scambio di modelli di simulazione. L'applicativo consentirà di caricare un file FMU, estrarne i metadati, configurare ed eseguire una simulazione, e visualizzare i risultati tramite un'interfaccia grafica.

Il vincolo principale che ha guidato le scelte progettuali è l'utilizzo esclusivo di Kotlin come linguaggio di sviluppo per l'intero stack, dal collegamento con la libreria nativa – scritta in C – fino all'interfaccia utente. Il sistema supporta tre piattaforme target: Linux x64, Windows x64 e macOS ARM64. Quest'ultima piattaforma ha richiesto una soluzione ad hoc per la ricompilazione degli FMU distribuiti senza binari compatibili con Apple Silicon, rappresentando uno degli aspetti tecnici più rilevanti del lavoro.

1.2 Background

1.2.1 Sviluppo di applicazioni full stack monolinguaggio

Prima di addentrarci nella descrizione del background del progetto è importante definire in maniera corretta cosa sono le applicazioni **full stack** e le differenze che emergono tra lo sviluppo di esse e quello di applicazioni più tradizionali. Le componenti delle applicazioni vengono divise in due macrocategorie

- **Frontend:** in questa sezione dell'applicativo sono gestiti tutti gli aspetti inerenti all'interfaccia utente: sia quelli legati all'aspetto grafico che quelli legati alla logica di interazione con essa.
- **Backend:** in questa sezione invece sono gestiti tutti gli aspetti legati all'esecuzione delle funzionalità dell'applicazione, inclusa l'elaborazione dei dati, la loro memorizzazione e la comunicazione con altre applicazioni se necessario.

Le applicazioni *full stack* dunque, comprendono entrambe le componenti. Lo sviluppo di questo tipo di applicazioni può includere l'utilizzo di molteplici linguaggi in base alle necessità, impiegando linguaggi progettati per lo sviluppo del backend e altri per il frontend. Questa tipologia di sviluppo rientra nella definizione di sviluppo *poliglotta*, dove per poliglotta si definiscono i processi con uno o più compiti che richiedono attivamente la modifica di artefatti utilizzando più di un

linguaggio [MCK⁺24]. Di fatto tale strategia di sviluppo risulta essere la più adottata al giorno d'oggi: uno studio condotto su 139 sviluppatori professionisti riporta una media di sette linguaggi per progetto, con oltre il 90% degli intervistati che riporta di avere problemi legati all'interazione tra linguaggi [MKL17]. È importante precisare come l'adozione di linguaggi diversi per produrre applicativi non è solo legato esigenze tecniche, ma anche ragioni socio-tecniche – come le preferenze ed esperienza del team – concettuali e di business, come la gestione di software già esistente (*software legacy*) oppure vincoli di tipo *vendor lock-in* [MCK⁺24]. Tuttavia la distinzione tra linguaggi per *frontend* e quelli per il *backend* al giorno d'oggi viene gradualmente persa, infatti diversi linguaggi inizialmente progettati per l'utilizzo in una sola area, vengono poi estesi anche all'altra. L'esempio più rilevante e dominante al giorno d'oggi è **JavaScript**, il quale inizialmente nasce come linguaggio per i browser, quindi dedicato al *frontend*, per poi evolversi oltre i confini del browser permettendone l'esecuzione anche sui server. Questo permette dunque la creazione di applicativi *full stack* monolinguaggio, una tendenza sempre più diffusa. Il vantaggio principale che questo comporta è l'utilizzo di un solo linguaggio per tutto lo stack, il quale porta a una minore curva di apprendimento e una maggiore efficienza per quanto riguarda l'impiego di programmatori. Infatti l'utilizzo di più linguaggi e il conseguente cambio di contesto tra essi ha un impatto non solo in termini economici, ma anche rispetto all'efficienza dei programmatori. Il costo cognitivo che deriva dal cambiare linguaggio utilizzato è noto in linguistica, dove è stato misurato un costo temporale nel passare da una lingua naturale a un'altra durante la comprensione. Un primo studio pilota ha indagato se un costo analogo sia misurabile anche nel passaggio tra linguaggi di programmazione nello stesso compito di sviluppo, sebbene i risultati ottenuti su un campione ridotto non abbiano permesso di confermare un effetto statisticamente significativo, lo studio lascia la questione aperta per future ricerche su scala maggiore [US18]. Uno dei più grandi limiti dello sviluppo *full stack* monolinguaggio è l'interazione con librerie e Application Programming Interface (API) native, infatti linguaggi come JavaScript sono noti per avere meccanismi limitati o comunque ostici per interagire con le librerie native, i quali saranno poi definiti nella sezione 1.2.1. In seguito verranno discusse in dettaglio le soluzioni esistenti per la programmazione di tale tipo di applicativi e i loro limiti, per poi analizzare il problema della portabilità e

dell'integrazione con le librerie native.

Panoramica delle soluzioni esistenti e loro limiti

Come anticipato in precedenza, esistono diversi linguaggi impiegati per la programmazione di app *full stack* monolinguaggio. Il più popolare attualmente risulta essere JavaScript¹, questo per via dello sviluppo della tecnologia **Node.js**; infatti questo permette di portare l'ambiente di esecuzione di JavaScript fuori dai browser web. Inoltre anche la creazione di TypeScript, una diretta evoluzione di JavaScript, sta contribuendo alla crescita di questo tipo di linguaggi. Tutti questi fattori rendono JavaScript una delle scelte più gettonate per la creazione di software *full stack*, impiegando diversi stack come Next.js, framework sviluppato da Vercel, e lo stack MEAN, il quale consiste in Mongo DB, Express.js e Angular come framework per il backend e frontend rispettivamente, il tutto legato da Node.js. Nonostante il grande supporto in termini di librerie e framework per questo tipo di tecnologia, l'integrazione con le librerie native risulta comunque ardua per via delle peculiarità del linguaggio.

Portabilità e integrazione con librerie native

Per librerie native si intendono tutte le librerie compilate in linguaggio macchina specifico per un architettura. Queste espongono delle **Application Binary Interface (ABI)** ovvero delle interfacce applicative che comunicano direttamente con il sistema operativo a livello di linguaggio macchina. Tra i linguaggi che vengono compilati in codice macchina figurano, C, C++, Rust e Zig; questi spesso vengono utilizzati per la realizzazione di programmi che hanno necessità di accedere direttamente all'hardware sottostante, oppure per sviluppare librerie che implementano alcuni standard. Quest'ultimo caso è il più rilevante per la tesi, infatti l'applicativo realizzato per questa tesi interagisce con dei file aderenti allo standard FMI, il quale verrà approfondito in seguito nella sezione dedicata.

Diverse delle soluzioni menzionate nella sezione precedente, richiedono la scrittura di codice collante tra il proprio applicativo e le librerie native qualora le si vogliano utilizzare. Nel caso più rilevante, ovvero JavaScript, se si necessita di

¹Survey realizzato da Stack Overflow delle tecnologie più utilizzate nel 2025.

interagire con librerie scritte in C++, occorre utilizzare una libreria C++ denominata **Node-API**. Tale libreria verrà poi utilizzata per creare moduli che fungono da collante tra JavaScript e la libreria nativa, assumendosi la responsabilità di effettuare le chiamate a quest'ultima². È proprio questo aspetto di interfacciamento con altri linguaggi che rappresenta la difficoltà più importante affrontata dagli sviluppatori *multilinguaggio*. Altri ostacoli sono spesso rappresentati dalla difficoltà di elaborazione dei dati tra diversi linguaggi e l'integrazione tra essi e la difficoltà nel processo di *building* [YLWC23]. La soluzione presentata per JavaScript permette dunque l'integrazione con le librerie native, ma contraddice direttamente il principio dietro lo sviluppo *full stack monolinguaggio*, rendendo necessaria la scrittura di codice in linguaggi al di fuori di quello scelto per la realizzazione dell'applicativo. Da questa analisi emerge un pattern ricorrente: le soluzioni *full stack monolinguaggio* più mature non sono state costruite attorno all'interoperabilità nativa come requisito principale. Le soluzioni legate a questi linguaggi dunque richiedono di violare il principio del monolinguaggio per interagire con le librerie native. Questo limite non dipende da aspetti implementativi risolvibili con nuove librerie, bensì una conseguenza strutturale basata sulla natura del linguaggio scelto. Dunque una reale soluzione richiederebbe un approccio diverso fin dalla scelta del linguaggio, rendendo necessario l'impiego di un linguaggio **multitarget**.

1.2.2 Kotlin e framework multitarget come risposta al gap

Alla luce delle limitazioni analizzate nella sezione precedente – in particolare la difficoltà di integrare librerie native C in ambienti JavaScript e la necessità di ricorrere a strati di collante eterolinguistici – questa sezione presenta Kotlin Multiplatform come risposta tecnologicamente motivata a tale gap. L'obiettivo non è descrivere la tecnologia e il suo funzionamento in modo esaustivo, ma illustrare perché le sue caratteristiche architetturali si adattano al problema in esame.

²Documentazione di Node.js per N-API.

Kotlin Multiplatform

Kotlin nasce nel 2011 come linguaggio sviluppato da JetBrains con l'obiettivo di modernizzare lo sviluppo sulla Java Virtual Machine (JVM): sintassi più concisa, sistema di tipi con null safety integrata e piena interoperabilità con Java [AB⁺20]. Queste caratteristiche non sono solo aspetti documentali, infatti studi empirici mostrano come tali caratteristiche vengano effettivamente adottate e usate dagli sviluppatori, riscontrando un uso di 15 su 26 delle nuove caratteristiche introdotte da Kotlin in almeno la metà delle applicazioni considerate nello studio, sottolineando come la diffusione di queste cresca insieme all'evoluzione delle applicazioni [MM20]. La maturità dell'ecosistema e il supporto di JetBrains – nonché l'adozione ufficiale da parte di Google come linguaggio primario per lo sviluppo Android – ne hanno consolidato la credibilità come linguaggio di produzione. La scelta di Kotlin come linguaggio non è solo motivata dal supporto istituzionale che sta ricevendo, ma trova riscontro anche in studi empirici che mostrano come l'adozione di Kotlin sia associata a una qualità del codice maggiore rispetto a gli applicativi sviluppati in Java [MM19].

Kotlin Multiplatform (KMP)³ è un sistema multitarget integrato nell'ecosistema Kotlin che permette di compilare lo stesso codice sorgente verso target eterogenei: JVM, JavaScript per il browser, e codice nativo tramite backend LLVM: un'infrastruttura di compilazione open-source che, tramite una rappresentazione intermedia portabile, consente di generare codice macchina nativo per diverse architetture. Il punto centrale è che KMP non è un framework di astrazione che interpone uno strato uniformante tra il codice e la piattaforma, infatti genera artefatti nativi distinti per ciascun target, mantenendo le caratteristiche di performance e interoperabilità proprie di ciascuno. Questa distinzione è rilevante rispetto ad approcci come React Native o Flutter, dove il codice condiviso viene eseguito in un runtime separato.

Il meccanismo che rende possibile la condivisione selettiva del codice è la dichiarazione `expect/actual`⁴. Nel modulo condiviso (`commonMain`) è possibile dichiarare una funzione o una classe come `expect`, specificandone la firma senza

³Documentazione Kotlin Multiplatform.

⁴Documentazione `expect/actual`.

fornirne l'implementazione. Ciascun modulo specifico di piattaforma fornisce poi la corrispondente dichiarazione **actual**, con un'implementazione che può sfruttare le API native del target. Questo meccanismo consente di separare nettamente la logica di business – che risiede nel modulo comune ed è condivisa tra tutti i target – dalle implementazioni dipendenti dalla piattaforma, senza rinunciare al controllo dei tipi a tempo di compilazione.

Dal punto di vista del gap identificato in precedenza, la caratteristica rilevante di KMP è la presenza del target **Kotlin/Native**: il compilatore produce binari nativi tramite LLVM, senza dipendenza da una JVM a tempo di esecuzione. Questo apre la possibilità di eseguire codice Kotlin su piattaforme embedded, su macOS e iOS, e in generale in qualsiasi contesto dove una JVM non è disponibile o desiderabile. Soprattutto, Kotlin/Native offre un meccanismo diretto per interfacciarsi con librerie C — aspetto che viene trattato nella sottosezione successiva.

1.2.3 Interoperabilità con C tramite Cinterop

Kotlin/Native include uno strumento dedicato all'interoperabilità con librerie C denominato **cinterop**⁵. Il suo funzionamento si basa sulla lettura degli header `.h` di una libreria C: a partire da questi, **cinterop** genera automaticamente un modulo Kotlin contenente le definizioni corrispondenti – funzioni, struct, enumerazioni e costanti – con tipi Kotlin appropriati. Lo sviluppatore configura il processo tramite un file `.def`, in cui specifica quali header includere, eventuali flag di compilazione, e parametri di mappatura dei tipi. Il tutto avviene a tempo di compilazione, producendo un modulo che può essere importato nel codice Kotlin come qualsiasi altra dipendenza.

Il confronto con Node-API, descritta nella sezione precedente, è diretto. Con Node-API l'integrazione di una libreria C in un progetto JavaScript richiedeva la scrittura manuale di codice C++ come strato di collegamento: il binding tra i tipi JavaScript e quelli C doveva essere gestito esplicitamente dallo sviluppatore, con tutti i costi di manutenzione e le insidie di type safety che ne derivano. Con **cinterop**, questo strato viene generato automaticamente dal compilatore a partire dagli header: lo sviluppatore scrive esclusivamente in Kotlin, senza mai uscire dal

⁵Documentazione **CInterop**.

proprio linguaggio di riferimento. Il principio del monolinguaggio – compromesso nell’approccio Node-API – viene quindi preservato.

Vale la pena segnalare un limite architetturale rilevante: `cinterop` è disponibile esclusivamente nel target Kotlin/Native. Il codice che utilizza collegamenti generati da `cinterop` non può quindi risiedere nel modulo `commonMain` di un progetto KMP, ma deve essere collocato nel modulo nativo specifico. In pratica, questo significa che le chiamate dirette alle librerie C costituiscono un’implementazione **actual** nel modulo nativo, mentre l’interfaccia esposta al codice comune viene dichiarata tramite la corrispondente dichiarazione **expect**. Non si tratta di una limitazione bloccante, ma di un vincolo di design che influenza la struttura del progetto e che verrà ripreso nella descrizione architetturale nei capitoli successivi.

1.2.4 Lo standard FMI e il formato FMU

La simulazione di sistemi dinamici complessi richiede spesso l’integrazione di componenti provenienti da strumenti e ambienti di sviluppo eterogenei, un problema noto in letteratura come co-simulazione [GTB⁺18], che indaga le tecniche per ottenere una simulazione globale tramite la composizione di simulatori indipendenti per le singole parti di un sistema. Lo standard FMI nasce proprio come una delle risposte più diffuse a questa esigenza, definendo un’interfaccia comune che consente lo scambio e l’esecuzione di modelli di simulazione indipendentemente dallo strumento con cui sono stati prodotti. Questa sezione si occupa dunque di presentare lo standard e il formato archivistico con cui viene condiviso, con l’obiettivo di fornire il contesto necessario a comprendere le scelte progettuali descritte nei capitoli successivi.

Functional Mock-up Interface (FMI 2.0)

FMI è uno standard aperto che definisce un’interfaccia C e un formato di impacchettamento per lo scambio di modelli di simulazione dinamica tra strumenti diversi. Lo sviluppo dello standard è stato coordinato da Daimler AG, che ora prende il nome di Mercedes-Benz Group AG, nell’ambito del progetto europeo ITEA2 MODELISAR con l’obiettivo di semplificare lo scambio di modelli di simulazione tra fornitori e case automobilistiche. La versione 1.0 è stata pubblicata nel

2010; la versione 2.0, che è quella rilevante per questo progetto, è stata rilasciata nel 2014 e ha raggiunto una diffusione molto ampia, con supporto da parte di oltre 280 strumenti di simulazione⁶.

L'idea centrale di FMI è la separazione netta tra la descrizione dell'interfaccia del modello – espressa in un file XML – e la sua implementazione computazionale, distribuita come libreria condivisa compilata in C. Questa separazione consente a qualsiasi strumento compatibile di importare e simulare un componente senza conoscere i dettagli interni del modello, né dipendere dallo strumento che lo ha generato.

Lo standard definisce due modalità operative distinte, che determinano come il componente viene integrato nel sistema simulante:

- **Model Exchange (ME):** l'FMU espone le equazioni del modello – tipicamente sotto forma di equazioni differenziali ordinarie – senza includere un risolutore numerico. È lo strumento importatore a fornire il risolutore e a gestire l'integrazione. Questa modalità offre maggiore controllo sulla precisione numerica ed è preferita quando il modello deve essere integrato strettamente in un ambiente di simulazione esistente.
- **Co-Simulation (CS):** l'FMU include il proprio risolutore numerico. Lo strumento importatore si limita a impostare gli ingressi, a richiedere all'FMU di avanzare di un passo temporale tramite la funzione `fmi2DoStep`, e a leggere le uscite al termine del passo. Questa modalità è la più diffusa in pratica, in quanto tratta il componente come una scatola nera e minimizza le dipendenze tra simulatore e modello. Sebbene la modalità *Co-Simulation* tratti l'FMU come una scatola nera dal punto di vista dello strumento importatore, l'uso di dello standard FMI in scenari di co-simulazione reali introduce problemi non banali di accuratezza numerica e di coordinamento del passo temporale tra i componenti, specialmente in presenza di dinamiche ibride che comprendono simulazioni con variabili discrete e continue [CLB⁺16]. Questa dinamica tuttavia va oltre lo scopo della presente tesi, la quale adotta un singolo modello FMU e un passo di integrazione fisso, ma evidenzia come

⁶Sito ufficiale dello standard FMI.

l'integrazione dello standard FMI non si riduce a un problema di mero collegamento *software*.

L'interfaccia C esposta dall'FMU segue una convenzione di denominazione prefissata: le funzioni hanno nomi nella forma `fmi2<NomeFunzione>` [Mod14], dove il prefisso `fmi2` identifica la versione dello standard. Il ciclo di vita di un'istanza FMU prevede fasi distinte – istanziamento, inizializzazione, simulazione, terminazione – ciascuna governata da specifiche chiamate di funzione. Questo contratto è ciò che garantisce la portabilità: qualunque strumento che rispetti la sequenza di chiamata prescritta dallo standard può utilizzare correttamente qualsiasi FMU conforme.

Struttura di un file FMU e modalità operative

Un file FMU è fisicamente un archivio ZIP rinominato con estensione `.fmu`.

La struttura interna è standardizzata e prevede i seguenti componenti principali [Mod14]:

- **modelDescription.xml**: è l'unico file obbligatorio. Contiene tutta la descrizione statica del modello: nome, identificatore, versione FMI, GUID di identificazione univoca, elenco delle variabili scalari con tipo, unità, valore iniziale e direzione (input, output, parametro, variabile di stato), e i flag che dichiarano quali funzionalità opzionali dello standard FMU supporta.
- **binaries/<piattaforma>/:** directory che contiene le librerie condivise compilate per ciascuna piattaforma target. La struttura delle sottodirectory segue la convenzione `<architettura>-<sistema operativo>`, ad esempio `x86_64-linux` o `x86_64-windows`. Un singolo file FMU può contenere binari per piattaforme multiple, garantendo portabilità senza ricompilazione.
- **sources/:** directory opzionale contenente il codice sorgente C del modello. La sua presenza consente la ricompilazione del componente per target non coperti dai binari precompilati — aspetto rilevante per ambienti embedded o architetture non standard.

- **resources/**: directory opzionale per dati accessori richiesti dal modello a tempo di esecuzione, come tabelle di lookup, file di configurazione o mappe.

Il file `modelDescription.xml` costituisce il contratto tra il produttore e il consumatore dell'FMU: definisce in modo dichiarativo e leggibile da macchina tutto ciò che il simulatore deve sapere per utilizzare correttamente il componente, prima ancora di caricare la libreria condivisa. In particolare, l'elenco delle *ScalarVariable*, con i relativi *valueReference*, ovvero gli identificatori numerici usati nelle chiamate `fmi2Get/fmi2Set`, costituisce la mappa tra i nomi simbolici delle variabili e le loro rappresentazioni interne.

Dal punto di vista di questo progetto, la struttura descritta pone una sfida concreta: accedere a un file FMU richiede di estrarre l'archivio ZIP, analizzare il file XML, e caricare dinamicamente la libreria condivisa appropriata per la piattaforma corrente. Queste operazioni vengono delegate a una libreria nativa C esistente, che gestisce l'interazione con il file FMU e ne astrae i dettagli di basso livello. Il punto rilevante per le scelte architetturali di questo progetto è che tale libreria espone un'interfaccia C, integrabile direttamente in Kotlin/Native tramite il meccanismo `cinterop` descritto nella sezione precedente.

Chapter 2

Analisi

Il presente capitolo descrive i requisiti funzionali del sistema e il modello del dominio, con particolare attenzione alle entità coinvolte e ai vincoli che ne regolano il comportamento.

2.1 Requisiti

Di seguito sono elencati i requisiti funzionali del sistema, ovvero le capacità che il sistema deve offrire all'utente o ai componenti che con esso interagiscono.

1. **Caricamento di un file FMU:** Il sistema deve consentire il caricamento di un file nel formato FMU (`.fmu`) tramite interfaccia web. Il file deve essere validato (estensione, formato) e salvato.
2. **Inizializzazione del modello:** Il sistema deve consentire l'inizializzazione del modello FMU precedentemente caricato attraverso l'utilizzo di una libreria nativa, per poi visualizzarne i metadati.
3. **Ispezione dei metadati:** Il sistema deve esporre le informazioni descrittive del modello FMU caricato, tra cui: nome del modello, autore, versione dello standard FMI, tipo di FMU (CS o ME), parametri dell'esperimento di default (istante di inizio, fine e passo), e lista delle variabili con la relativa unità di misura.

4. **Esecuzione della simulazione:** Il sistema deve consentire di avviare una simulazione Co-Simulation sul modello caricato, accettando in input una configurazione (istante di inizio, istante di fine, passo di integrazione, e lista di variabili di output).
5. **Visualizzazione dei risultati:** Il frontend deve presentare i risultati della simulazione in forma grafica, tramite grafici a linea che rappresentino l'andamento temporale delle variabili selezionate.
6. **Selezione delle variabili di output:** L'utente deve poter selezionare un sottoinsieme delle variabili disponibili da includere nell'output della simulazione.
7. **Ricompilazione di FMU con sorgenti (macOS):** Su piattaforma macOS ARM64, il sistema deve essere in grado di compilare FMU che includono i file sorgente C, producendo una libreria universale compatibile con le architetture arm64 e x86-64.
8. **Pulizia delle risorse:** Al termine il sistema deve liberare le risorse allocate (file temporanei, librerie native) in modo sicuro e deterministico.

2.2 Modello del dominio

Il sistema opera su modelli di simulazione distribuiti nel formato FMU. Il flusso principale prevede che un file FMU venga fornito al sistema, che ne estrae i metadati e li rende disponibili, e che sulla base di questi sia possibile avviare una simulazione e ottenerne i risultati.

Un **modello FMU** è il file che viene condiviso al sistema. All'interno ha diverse componenti, tra cui il file che contiene l'insieme di metadati descrittivi – nome, autore, versione, tipo – e da un insieme di **variabili scalari**, ciascuna con un nome e un'unità di misura. Il tipo del modello determina le operazioni che il sistema può svolgere su di esso: i modelli *Co-Simulation* supportano l'esecuzione di simulazioni, mentre dei modelli *Model Exchange* si possono semplicemente visualizzare i dati.

Una **simulazione** è l'esecuzione di un modello Co-Simulation in un intervallo temporale definito da un istante di inizio, un istante di fine e un passo di inte-

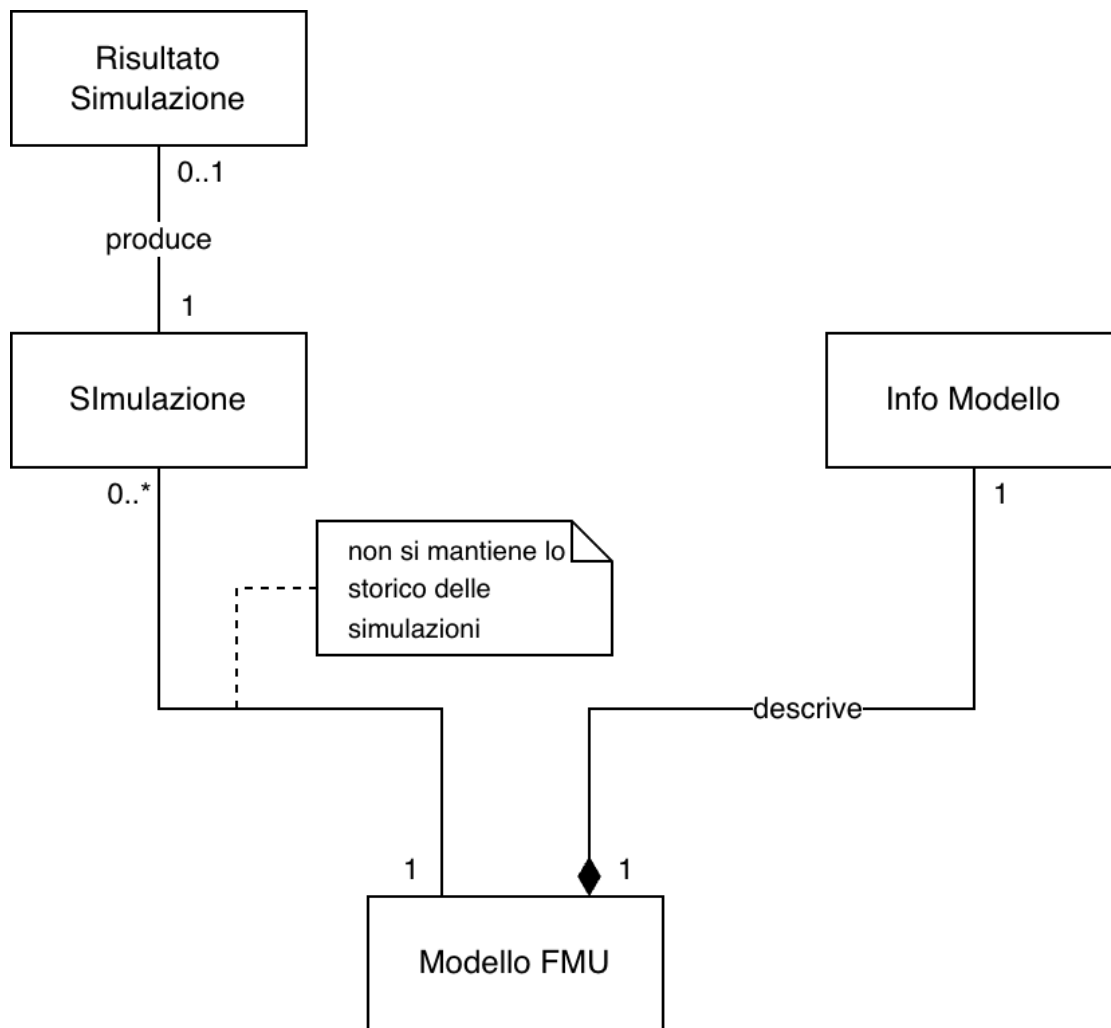


Figure 2.1: Schema rappresentate le entità del modello del dominio.

grazione. È possibile specificare un sottoinsieme delle variabili del modello da osservare; se non indicato, vengono registrate tutte le variabili disponibili.

Il **risultato** di una simulazione è la serie temporale prodotta: per ogni istante campionato, il valore assunto da ciascuna variabile osservata. Il risultato mantiene riferimento alla configurazione che lo ha generato.

Il sistema gestisce un solo modello alla volta. Il caricamento di un nuovo file FMU sostituisce il modello precedente e invalida eventuali stati associati a esso. La figura 2.1 rappresenta le entità del modello dell'applicativo.

2.2.1 Vincoli

I vincoli di progetto delimitano lo spazio entro cui il sistema deve essere realizzato.

- **Monolinguaggio:** l'intero sistema deve essere sviluppato in un unico linguaggio di programmazione, senza ricorrere a codice collante in altri linguaggi per l'integrazione con le librerie native — vincolo che, come discusso nel background, esclude le soluzioni full stack tradizionali basate su JavaScript.
- **Compatibilità FMI 2.0:** il sistema deve operare su file FMU conformi allo standard FMI 2.0. Le versioni precedenti o successive dello standard non sono considerate.
- **Portabilità multiplatforma:** il sistema deve essere eseguibile su Linux, macOS e Windows senza modifiche al codice sorgente.
- **Utilizzo di una libreria nativa esistente:** l'interazione con i file FMU deve avvenire tramite una libreria C preesistente che implementa lo standard FMI. Non è previsto lo sviluppo di un'implementazione propria del protocollo.

Chapter 3

Design

3.1 Architettura

Il sistema è strutturato secondo un'architettura client-server a tre livelli – frontend, backend e logica di dominio – realizzati come moduli indipendenti all'interno di un unico progetto multimodulo¹. A questi si aggiunge il modulo `fmilib`, che non costituisce un livello architetturale in senso stretto ma un modulo di supporto alla build, responsabile della compilazione della libreria nativa. I quattro moduli che compongono il progetto sono quindi `fmilib`, `fmukt`, `backend` e `frontend`, ciascuno con responsabilità ben delimitate e dipendenze unidirezionali.

Il modulo `fmilib` costituisce la base della catena di dipendenze: si occupa della compilazione della libreria nativa *FMILib*² e ne espone gli artefatti (header C e librerie condivise) agli strati superiori. La scelta di rendere questo modulo indipendente isola completamente la toolchain di compilazione dal resto del progetto Kotlin, rendendo il confine tra codice nativo e codice multiplatforma esplicito e controllato.

Il modulo `fmukt` costituisce il cuore della logica di dominio. Dipende da `fmilib` e si occupa di tutto ciò che riguarda il ciclo di vita degli FMU: caricamento del file, parsing dell'XML descrittivo, invocazione della libreria nativa per l'esecuzione della simulazione, e restituzione dei risultati. Il modulo è com-

¹GitHub contenente il codice del progetto.

²GitHub contenente il codice della libreria FMILib.

pilato come libreria Kotlin Multiplatform per le piattaforme native (Linux x64, macOS ARM64 e Windows x64), consentendo di riutilizzare la logica comune tra le piattaforme e di isolare, nei codici sorgente specifici, le parti che richiedono comportamenti differenziati: in particolare la fase di preelaborazione degli FMU su macOS ARM, che richiede la ricompilazione del modello.

Il modulo **backend** implementa un server HTTP REST che espone le funzionalità di **fmu-kt** come API consumabile dal frontend, è anch'esso un modulo Kotlin Multiplatform con target nativi. Il server espone endpoint per il caricamento del file FMU, l'inizializzazione, la lettura dei metadati e l'esecuzione della simulazione.

Il modulo **frontend** realizza l'interfaccia utente web. Il frontend comunica con il backend, permettendo così all'utente di interagire con gli endpoint esposti e di avere una visualizzazione grafica delle informazioni e dei risultati della simulazione. Questa catena di interazioni è rappresentata nella figura 3.1.



Figure 3.1: Diagramma rappresentante relazione tra i moduli dell'applicativo.

Questa scelta disaccoppia completamente i tre processi: il backend può essere avviato indipendentemente, testato in isolamento tramite client HTTP standard, e in futuro potrebbe essere esposto su rete senza modifiche architetturali. Il frontend non conosce nulla della logica FMU; conosce solo i contratti delle API REST. Mentre il modulo **fmu-kt** funge da libreria indipendente, dunque può essere esportata per essere usata in altri progetti.

Una conseguenza rilevante di questa architettura è la separazione netta tra la parte del sistema che richiede accesso nativo – **fmu-kt** e **backend**, che devono interagire con la libreria C e con il filesystem – e la parte puramente applicativa del frontend, che rimane nel dominio Kotlin puro. Questo ha guidato la decisione di non integrare **fmu-kt** direttamente nel frontend, evitando di contaminare il modulo che implementa la User Interface (UI) con dipendenze native.

3.2 Design di dettaglio

Le sezioni seguenti approfondiscono le scelte di design dei tre componenti principali del sistema, seguendo lo stesso ordine delle dipendenze architetturali descritte in precedenza: si parte dall'interfacciamento con le librerie native, che costituisce il fondamento su cui si appoggiano gli strati superiori, per risalire al backend, che orchestra la logica di dominio ed espone i servizi verso l'esterno, fino al frontend, responsabile della presentazione e dell'interazione con l'utente.

Per ciascun componente vengono discusse le scelte progettuali significative, con particolare attenzione ai confini tra codice condiviso e codice specifico per piattaforma, e alle motivazioni che hanno guidato la separazione delle responsabilità.

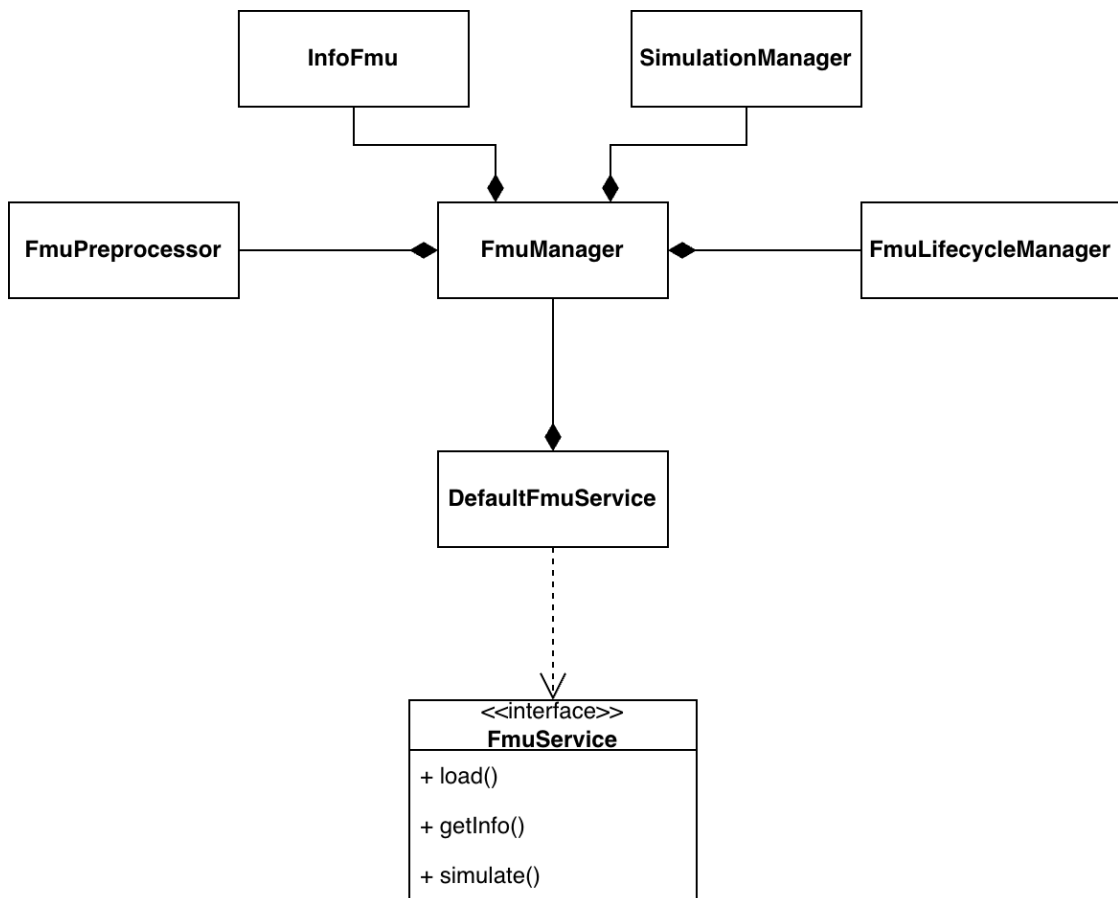
3.2.1 Interfacciamento con librerie native

Il modulo `fmu-kt` è il componente del sistema responsabile di tutta l'interazione con la libreria nativa FMILib. Il suo design è orientato ad astrarre la complessità di basso livello esposta dalla libreria C, presentando agli strati superiori un'interfaccia di alto livello che non richiede alcuna conoscenza dei dettagli nativi.

Il punto di accesso esterno al modulo è `FmuService`, un'interfaccia che espone le operazioni disponibili sugli FMU: caricamento, lettura dei metadati e avvio della simulazione. Questa scelta consente al backend di dipendere esclusivamente dal contratto dell'interfaccia, rendendo il modulo sostituibile e facilmente testabile tramite implementazioni fittizie. Ulteriori funzionalità potranno essere introdotte in futuro aggiungendo nuovi metodi all'interfaccia e le relative implementazioni, senza alterare la struttura generale del modulo.

L'implementazione concreta del servizio, denominata `DefaultFmuService`, delega la gestione del modello a `FmuManager`, un componente che orchestra l'intero ciclo di vita di un FMU attraverso tre manager specializzati, ciascuno con una responsabilità ben delimitata:

- **FmuLifecycleManager**: si occupa dell'apertura e della chiusura dell'FMU, gestendo l'allocazione e il rilascio delle risorse associate.
- **InfoManagerFmu**: estrae i metadati del modello – nome, autore, versione,

Figure 3.2: Diagramma delle dipendenze di `DefaultFmuService`

variabili disponibili e parametri dell’esperimento predefinito – interrogando direttamente le strutture dati native.

- **SimulationManager:** gestisce la configurazione e l’esecuzione della simulazione raccogliendo i risultati e restituendoli in un formato indipendente dalla libreria nativa.

La figura 3.2 rappresenta con un diagramma le dipendenze di `DefaultFmuService`.

Questi tre manager sono gli unici componenti del sistema che interagiscono direttamente con FMILib. Concentrare questo confine in un insieme ristretto e ben identificato di classi rende il sistema più manutenibile: eventuali cambiamenti nell’API nativa o l’adozione di una libreria FMI alternativa richiederebbero modifiche localizzate nei manager, senza propagarsi al resto dell’architettura.

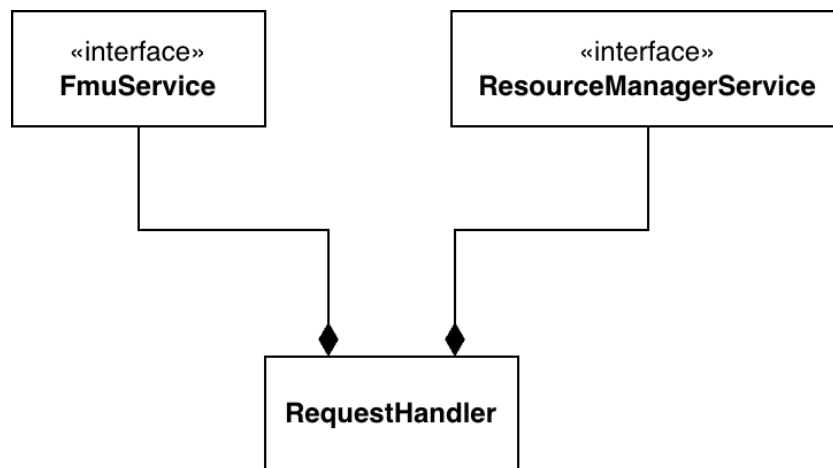
Prima che un FMU possa essere caricato, è necessaria una fase di preparazione del file che ne garantisce la compatibilità con la piattaforma corrente. Questa responsabilità è affidata al `FmuPreprocessor`, un componente separato la cui implementazione varia a seconda della piattaforma target. La sua interfaccia è definita nel source set comune, mentre le implementazioni specifiche risiedono nei source set di piattaforma, seguendo il meccanismo `expect/actual` di Kotlin Multiplatform. In questo modo la logica di preparazione dell'FMU rimane completamente trasparente agli strati superiori.

3.2.2 Backend multiplatforma

Il backend è strutturato in modo da separare nettamente la definizione delle rotte HTTP dalla logica applicativa che ne gestisce le richieste. Il componente centrale di questa separazione è `RequestHandler`, che raccoglie tutta la logica di gestione delle richieste e viene iniettato nel layer di routing al momento dell'avvio del server. In questo modo le rotte rimangono dichiarative e di facile lettura, limitandosi a descrivere la mappatura tra endpoint e operazioni, mentre tutta la logica risiede in un unico posto ben identificato. Aggiungere o modificare un endpoint richiede quindi interventi localizzati e indipendenti tra loro.

`RequestHandler` è a sua volta una composizione di due componenti, ciascuno con una responsabilità distinta:

- **ResourceManagerService**: è l'interfaccia che definisce le operazioni di gestione del filesystem locale, in particolare del salvataggio del file FMU ricevuto dal client e della restituzione dei percorsi necessari al caricamento del modello. Isolare questa responsabilità in un componente dedicato rende esplicito il confine tra la gestione delle risorse persistenti e la logica di interazione con il modello, e semplifica la sostituzione della strategia di memorizzazione, ad esempio in fase di test.
- **FmuService**: è l'interfaccia del modulo `fmukt` descritta nella sezione precedente. Il backend non dipende da nessun dettaglio interno di quel modulo: conosce solo le operazioni che il servizio espone, mantenendo il disaccoppiamento tra i due moduli.

Figure 3.3: Design di `RequestHandler`.

Il diagramma dei componenti appena elencati e raffigurato nella figura 3.3.

Questa composizione rispecchia il flusso naturale di una richiesta: quando il client avvia una simulazione, `RequestHandler` gestisce il caricamento del file, che viene salvato dal `ResourceManagerService` e successivamente fornito a `FmuService` per il caricamento e l'esecuzione del modello. Le due responsabilità – dove si trova il file e cosa farne – restano separate e intercambiabili indipendentemente. La scelta far dipendere `RequestHandler` da un'interfaccia piuttosto che a una classe, per quanto riguarda la gestione del filesystem, è motivata dalla volontà di astrarre l'organizzazione interna dei file, permettendo così di realizzare un sistema di salvataggio organizzato in modo diverso senza intaccare in alcun modo il `RequestHandler`.

3.2.3 Frontend multiplatforma

Il frontend è organizzato attorno a due responsabilità principali, riflesse nella struttura del modulo: la presentazione dell'interfaccia utente e la comunicazione con il backend.

La navigazione tra le diverse viste dell'applicazione è gestita attraverso il concetto di *screen*, ognuno dei quali rappresenta una schermata autonoma con una funzione ben precisa. Sono presenti una schermata iniziale (`HomeScreen`), una per la visualizzazione dei metadati del modello caricato (`InfoScreen`), una per la

configurazione e l'avvio della simulazione (`SimulationScreen`) e una per la visualizzazione grafica dei risultati (`ChartScreen`). Questa suddivisione rende il codice dell'interfaccia facilmente navigabile e consente di aggiungere nuove schermate senza interferire con quelle esistenti.

La comunicazione con il backend è invece centralizzata nel modulo `client`, che espone le funzioni necessarie per interagire con ciascun endpoint REST. Concentrare questa logica in un unico punto evita la duplicazione del codice di comunicazione tra le schermate e isola i dettagli del protocollo HTTP dal codice dell'interfaccia utente. Le schermate si limitano quindi a invocare le funzioni del client e a reagire ai risultati, senza conoscere nulla della comunicazione sottostante.

Chapter 4

Implementazione

Questo capitolo descrive le scelte implementative principali del sistema, seguendo un percorso che va dagli strati più bassi verso quelli più alti, in modo da costruire progressivamente il contesto necessario alla comprensione di ciascuna parte.

Si parte dal binding tra Kotlin e la libreria nativa FMILib, illustrando come il meccanismo di interoperabilità con il codice C sia stato configurato e come le opzioni del linker siano state adattate per ciascuna piattaforma target. Su questa base si descrive poi la gestione del ciclo di vita dell'FMU, ovvero come il sistema allochi il contesto FMI, effettui il parsing dell'XML descrittivo e carichi il binario nativo. Si procede quindi con l'estrazione dei metadati e la gestione delle variabili, spiegando come le informazioni del modello vengano interrogate tramite FMILib e restituite in un formato indipendente dalla libreria. La sezione sull'esecuzione della simulazione descrive invece il loop di avanzamento temporale, la risoluzione dei value reference e la raccolta dei risultati a ogni passo.

Una sezione dedicata tratta la ricompilazione degli FMU su macOS ARM64, un caso particolare richiesto dall'architettura Apple Silicon, in cui i binari distribuiti negli FMU non sono compatibili con la piattaforma e devono essere ricompilati a partire dai sorgenti C inclusi nel pacchetto. Il capitolo si chiude con una panoramica degli strumenti di processo adottati: la configurazione Gradle multipiattaforma, la pipeline di CI/CD e il workflow di build e test su Linux, macOS e Windows.

4.1 Binding C/Kotlin e configurazione del linker per piattaforma

L'interoperabilità tra Kotlin e la libreria nativa FMILib è realizzata tramite il meccanismo di C interop di Kotlin/Native, che genera automaticamente binding Kotlin a partire dagli header C della libreria. La configurazione del binding avviene tramite un file `.def` in cui vengono dichiarati gli header da includere e le opzioni di compilazione e linking necessarie, tra cui il percorso alla directory degli header e il nome della libreria condivisa da collegare. A partire da questa dichiarazione, il compilatore produce un modulo Kotlin che espone le funzioni e i tipi C come API direttamente utilizzabili dal codice Kotlin, boilerplate manuale minimo rappresentato dalla scrittura del file `.def`.

La libreria FMILib non è disponibile come artefatto precompilato su repository pubblici, ma viene compilata a partire dai sorgenti tramite CMake. La compilazione è gestita dal modulo `fmilib`, che si occupa di invocare CMake, eseguire l'installazione degli artefatti e renderli disponibili al resto del progetto attraverso una configurazione e un plugin Gradle dedicato. Il modulo `fm-kt` dichiara una dipendenza su questi artefatti e li riferenzia nelle opzioni di linker del target nativo, includendo sia la directory degli header sia quella delle librerie condivise prodotte dalla compilazione.

Poiché il sistema deve essere compilato su tre piattaforme distinte: Linux x64, macOS ARM64 e Windows x64, le opzioni di linker sono configurate per piattaforma, in quanto i percorsi degli artefatti e le convenzioni del linker differiscono tra i sistemi operativi. Su Windows, ad esempio, la libreria condivisa è un file `.dll` che deve essere copiato nella stessa directory dell'eseguibile prodotto; su Linux e macOS si tratta invece di file `.so` e `.dylib` riferenziabili tramite `rpath`.

Su Linux si è resa necessaria un'ulteriore soluzione alternativa legata alla toolchain interna di Kotlin/Native. Il compilatore Konan, utilizzato da Kotlin/Native, include un proprio sysroot con una versione di glibc datata (2.19, kernel 4.9), pensata per massimizzare la compatibilità tra distribuzioni Linux¹. Quando la libreria da collegare è stata compilata con una versione più recente di glibc, come

¹Discussione mismatch glibc del compilatore.

accade negli ambienti CI moderni, il linker di Konan non riesce a risolvere alcuni simboli di sistema presenti nella libreria condivisa di FMILib. Per ovviare a questo problema si è fatto ricorso alle opzioni `--allow-shlib-undefined` e `--unresolved-symbols=ignore-all`, che istruiscono il linker a non fallire in presenza di simboli non risolti nelle librerie condivise, delegandone la risoluzione al linker dinamico del sistema operativo a tempo di esecuzione. Non esiste una soluzione ufficiale da parte di JetBrains: il workaround adottato è quello consolidato nella comunità², ed è accettabile in quanto i simboli in questione sono garantiti essere presenti in qualsiasi distribuzione Linux moderna su cui il binario verrà eseguito.

4.2 Gestione del ciclo di vita dell'FMU

Un FMU è distribuito come archivio ZIP con estensione `.fmu`, contenente un file XML di descrizione del modello (`modelDescription.xml`), i binari precompilati per le piattaforme supportate e, opzionalmente, i sorgenti C del modello. Prima che qualsiasi operazione possa essere eseguita sul modello, l'archivio deve essere estratto in una directory di lavoro: questa operazione è delegata direttamente a FMILib, a cui vengono forniti il percorso del file `.fmu` e la directory di destinazione dell'estrazione. La libreria si occupa autonomamente di estrarre il contenuto e di rendere disponibili i file necessari ai passi successivi.

Una volta estratto il contenuto, FMILib costruisce un contesto FMI: una struttura opaca che rappresenta lo stato interno della sessione nella libreria, per poi procedere all'analisi del file `modelDescription.xml`. Da questo file vengono estratti tutti i metadati del modello: nome, autore, versione, tipo di FMU (ME, CS o entrambi), i parametri predefiniti dell'esperimento e l'elenco delle variabili disponibili. Il risultato del parsing è una struttura dati interna a FMILib, referenziata tramite un puntatore opaco di tipo `fmi2_import_t`, che viene mantenuta per tutta la durata della sessione e liberata esplicitamente alla chiusura.

Il passo successivo è il caricamento del binario nativo del modello: FMILib individua la libreria condivisa appropriata per la piattaforma corrente all'interno

²Soluzione per il mismatch della glibc proposta da un utente.

dell'archivio estratto e la carica dinamicamente tramite le API del sistema operativo. Questo passaggio può fallire se il binario non è presente o non è compatibile con la piattaforma — caso che si presenta, ad esempio, su macOS ARM64 con FMU che distribuiscono solo binari x86_64, e che viene gestito tramite la ricompilazione descritta nella Sezione 4.5.

Il ciclo di vita del modello si chiude con una fase di terminazione altrettanto esplicita: alla chiusura, il binario viene scaricato, la struttura FMI liberata e il contesto deallocato. Questa sequenza è incapsulata in `FmuLifecycleManager`, che implementa `AutoCloseable` per garantire che le risorse native vengano sempre rilasciate correttamente, anche in presenza di eccezioni.

Per quanto riguarda il ciclo di vita della simulazione, esso si innesta su quello del modello e segue le fasi definite dallo standard FMI 2.0 per la Co-Simulation: istanziamento del componente, configurazione dell'esperimento con i parametri temporali e di tolleranza, ingresso e uscita dalla modalità di inizializzazione, esecuzione dei passi di simulazione e terminazione. Queste operazioni sono gestite da `SimulationManager`, che riceve in ingresso la struttura `fmi2_import_t` già inizializzata da `FmuLifecycleManager` e opera su di essa per tutta la durata dell'esperimento. L'accesso alle variabili del modello durante l'esecuzione, quindi la loro risoluzione tramite value reference e la lettura dei valori a ogni passo, è l'argomento della sezione seguente.

4.3 Estrazione dei metadati e gestione delle variabili

L'estrazione dei metadati del modello avviene interamente tramite le API di `FMILib`, che espongono funzioni dedicate per interrogare la struttura `fmi2_import_t` prodotta dal parsing dell'XML. Per ciascun campo — nome del modello, autore, versione, tipo di FMU e parametri predefiniti dell'esperimento — esiste una corrispondente funzione C della libreria che restituisce il valore direttamente dalla struttura interna, senza richiedere un nuovo accesso al file XML. Questi valori vengono raccolti da `InfoManagerFmu` e assemblati in un oggetto `FmuInfo`, una data class Kotlin serializzabile che costituisce il contratto tra il modulo `fmu-kt` e

il backend.

La gestione delle variabili richiede un passo aggiuntivo rispetto ai semplici campi scalari. FMILib espone la lista delle variabili del modello come una struttura iterabile, da cui è possibile estrarre per ciascuna variabile il nome e il tipo base. Per le variabili di tipo reale viene anche estratta l'unità di misura, ottenuta navigando il puntatore all'unità associata alla variabile; per le variabili di tipo diverso l'unità non è applicabile e viene registrata come assente. Il risultato è una mappa da nome di variabile a unità di misura, inclusa in `FmuInfo`, che il frontend utilizza per presentare all'utente le variabili disponibili prima di avviare una simulazione. Una volta terminate le operazioni sulla lista, la struttura viene esplicitamente liberata tramite l'apposita API di FMILib, in linea con il modello di gestione manuale della memoria della libreria.

Tra i campi di `FmuInfo` figura anche il flag `canSimulate`, che indica se il modello supporta la modalità Co-Simulation. Questo valore viene derivato dal tipo di FMU rilevato durante il parsing: solo i modelli di tipo Co-Simulation o misto (Co-Simulation e Model Exchange) sono simulabili direttamente tramite FMILib, mentre i modelli di tipo Model Exchange puro richiederebbero un solver numerico esterno non previsto dal sistema.

4.4 Esecuzione della simulazione

L'esecuzione della simulazione segue la sequenza di stati definita dallo standard FMI 2.0 per la modalità Co-Simulation. Prima che il loop di simulazione possa essere avviato, il componente deve essere istanziato tramite una chiamata a `fmi2_import_instantiate`, che crea un'istanza del modello associandola a un nome di esperimento e specificando la modalità Co-Simulation. L'istanziamento può fallire – ad esempio se il binario non è stato caricato correttamente – e in tal caso viene sollevata un'eccezione che interrompe il flusso prima di entrare nella fase di configurazione.

Una volta istanziato il componente, viene invocata `fmi2_import_setup_experiment` con i parametri contenuti in `SimulationConfig`: tempo di inizio, tempo di fine, passo temporale e tolleranza numerica. I parametri opzionali – tempo di fine e tolleranza – vengono passati alla libreria insieme

a un flag booleano che ne indica la validità, seguendo la convenzione dell'API C di FMILib. Il modello entra quindi in modalità di inizializzazione tramite `fmi2_import_enter_initialization_mode` e ne esce con `fmi2_import_exit_initialization_mode`, completando la fase di setup e portando il modello nello stato pronto per la simulazione.

Prima di avviare il loop temporale, `SimulationManager` risolve i *value reference* delle variabili da campionare. Un *value reference* è un identificatore numerico che è associato a ciascuna variabile del modello e che viene utilizzato nelle chiamate di lettura della variabile durante la simulazione. La risoluzione avviene una sola volta prima del loop: per ogni variabile da registrare, il nome viene passato a `fmi2_import_get_variable_by_name` per ottenere il puntatore alla variabile, da cui si estrae il *value reference* tramite `fmi2_import_get_variable_vr`. I *value reference* vengono quindi disposti in un vettore, pronto per essere passato alle successive chiamate di lettura.

Le variabili da campionare sono determinate dal campo `outputVariables` di `SimulationConfig`: se l'utente ne ha specificate alcune, solo quelle vengono registrate; se la lista è vuota, vengono campionate tutte le variabili del modello. Questa scelta consente di limitare la quantità di dati prodotti dalla simulazione nei casi in cui l'utente sia interessato solo a un sottoinsieme delle variabili disponibili.

Il loop di simulazione avanza per passi temporali da `startTime` fino a `stopTime`. A ogni iterazione viene invocata `fmi2_import_do_step`, che fa avanzare il modello di un passo temporale. Il passo effettivo applicato è il minore tra il passo configurato e il tempo rimanente fino alla fine della simulazione, in modo da non sfiorare il tempo di stop anche quando il passo configurato non divide esattamente l'intervallo. Dopo ogni passo, i valori correnti delle variabili vengono letti con una singola chiamata a `fmi2_import_get_real`, che popola l'array di valori in memoria precedentemente allocato. Il timestamp corrente e i valori letti vengono quindi accodati alle rispettive liste di risultati.

Al termine del loop, il modello viene terminato tramite `fmi2_import_terminate` e l'istanza liberata con `fmi2_import_free_instance`. Queste operazioni avvengono in un blocco `finally`, garantendo che le risorse vengano rilasciate anche in caso di errore durante la simulazione. I risultati raccolti vengono restituiti come oggetto `SimulationResult`, una data class serializzabile

contenente la lista dei timestamp, la mappa da nome di variabile a lista di valori campionati, e la configurazione usata per produrli — quest’ultima inclusa per consentire al frontend di ricostruire il contesto dell’esperimento senza dover mantenere stato aggiuntivo.

4.5 Ricompilazione degli FMU su macOS ARM64

Lo standard FMI 2.0 non distingue tra architetture diverse all’interno della stessa famiglia di sistema operativo: per macOS è prevista un’unica cartella `darwin64` all’interno della directory `binaries` dell’archivio FMU, indipendentemente dall’architettura del processore. Di conseguenza, FMILib cerca il binario del modello esclusivamente in quella cartella, senza differenziare tra `x86_64` e `ARM64`. La soluzione naturale a questo vincolo dello standard è produrre un *fat binary*, ossia una libreria `.dylib` universale che incorpora entrambe le architetture, in modo che il binario risultante sia caricabile correttamente da FMILib su qualsiasi Mac, indipendentemente dall’architettura del processore host.

Tuttavia, la maggior parte degli FMU disponibili distribuisce nella cartella `darwin64` solo un binario compilato per `x86_64`, architettura standard prima che Apple Silicon diventasse una piattaforma diffusa. Questi binari non sono eseguibili nativamente su `ARM64` e, poiché FMILib li carica dinamicamente con `dlopen`, non è possibile affidarsi allo strumento Rosetta 2, incluso nelle distribuzioni MacOS moderne, per la traduzione. La soluzione adottata è ricompilare il binario del modello a partire dai sorgenti C inclusi nell’archivio FMU – solitamente presenti nella cartella `sources` – producendo due oggetti distinti per `x86_64` e `ARM64` e unendoli in un unico fat binary tramite `lipo`. Il fat binary viene quindi collocato nella cartella `darwin64`, sostituendo il binario originale, prima che FMILib proceda al caricamento.

Per realizzare questa pipeline di ricompilazione è stato necessario costruire un’infrastruttura di supporto composta da tre componenti, tutti residenti nel source set dedicato a macOS ARM64 in quanto specifici di quella piattaforma: `FileSystemManager`, che si occupa delle operazioni sul filesystem necessarie alla

pipeline (navigazione della directory dei sorgenti, individuazione dei file `.c`, scrittura del fat binary nella destinazione corretta); `ProcessExecutor`, che gestisce l'invocazione di processi di sistema come `lipo` e `clang`, catturandone l'output e il codice di uscita; e `ClangCompiler`, che orchestra le invocazioni al compilatore tramite `ProcessExecutor`. Isolare questi componenti nel source set macOS ARM64 è un vantaggio diretto del modello Kotlin Multiplatform: il codice specifico della piattaforma resta circoscritto ai sorgenti che lo riguardano, senza inquinare il source set comune.

La compilazione di ciascun file sorgente avviene in due passaggi distinti, uno per architettura. I flag di compilazione includono `-dynamiclib` per produrre una libreria condivisa e l'indicazione dell'architettura target tramite `-arch x86_64` e `-arch arm64` rispettivamente. Un aspetto non banale della compilazione riguarda gli header: i sorgenti C degli FMU includono tipicamente gli header FMI 2.0 (`fmi2Functions.h`, `fmi2FunctionTypes.h`, `fmi2TypesPlatform.h`), che non sono presenti nel sistema ma sono forniti dallo standard e devono essere resi disponibili al compilatore. La soluzione adottata è sintetizzare gli header che raccolgono le dichiarazioni necessarie e passarne il percorso a Clang tramite il flag `-I`. Non esiste un approccio canonico consolidato per questo problema nel contesto della ricompilazione runtime di FMU: la sintesi dell'header, gestita dalla classe `FmiHeaderSynthesiser`, è una soluzione pragmatica che elimina la dipendenza da un'installazione esterna degli header FMI sul sistema.

Le componenti descritte vengono orchestrate da una classe denominata `FmuRecompiler` raffigurata in 4.1.

Un ulteriore problema emerge durante il linking ed è intrinseco alla modalità di distribuzione degli FMU con sorgenti C. Lo standard FMI 2.0 prevede che, quando un FMU è distribuito con codice sorgente, i nomi delle funzioni vengano costruiti anteposendo il prefisso `FMI2_FUNCTION_PREFIX`, definito come il `modelIdentifier` dell'FMU seguito da `"_"`, al nome standard della funzione: una funzione come `fmi2DoStep` diventa quindi `ModelName_fmi2DoStep`. Questa convenzione serve a evitare conflitti tra simboli di FMU diversi quando più modelli vengono compilati e linkati staticamente nello stesso processo. `FMIlib`, tuttavia, si aspetta di trovare nella `.dylib` i simboli con i nomi standard non prefissati, poiché per le shared library lo standard prescrive l'assenza del prefisso. Per risolvere questa discrepanza

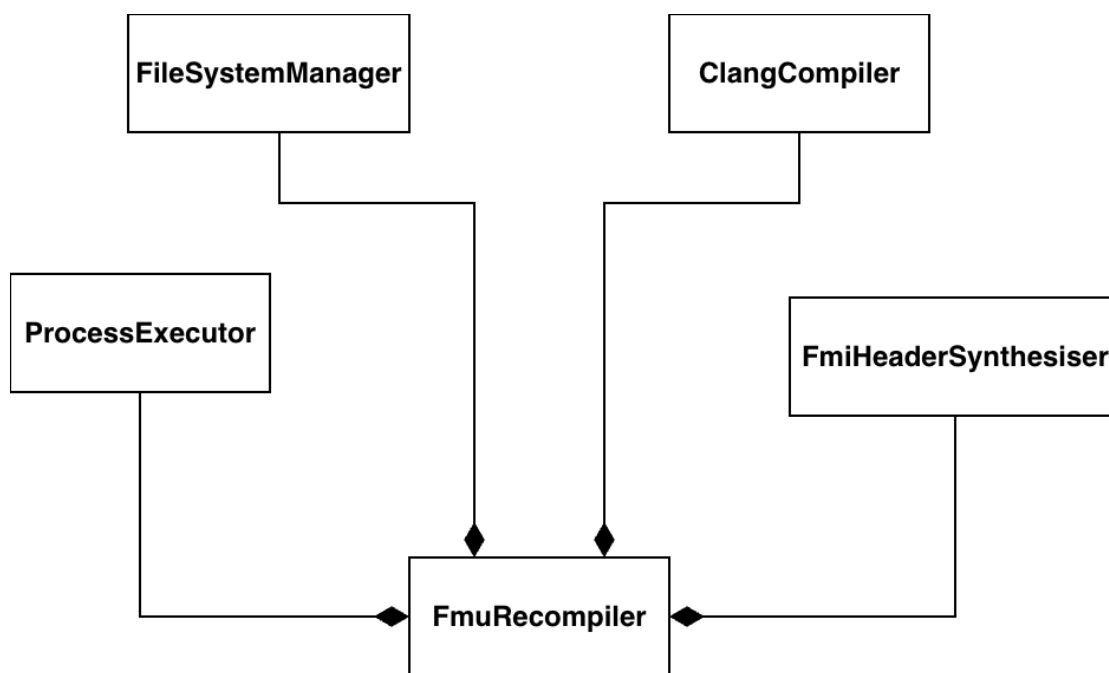


Figure 4.1: Struttura di FmuRecompiler

vengono creati degli alias a livello di linker tramite il flag `-alias`, che istruisce il linker a esporre ciascun simbolo prefissato anche con il nome standard atteso da FMILib, senza duplicare il codice.

4.6 Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma

Build system e configurazione Gradle L'intero applicativo è organizzato come un progetto Gradle multi-modulo, in cui ogni modulo – `fmilib`, `fmu-kt`, `backend` e `frontend` – dispone di un proprio `build.gradle.kts` con le dipendenze e i target specifici. Il build file radice si limita a dichiarare i plugin condivisi e i repository comuni, delegando tutta la logica ai moduli figli. Inoltre si è prediletto un approccio che isola il processo di build e compilazione della libreria nativa per via della complessità aggiuntive introdotte da sistemi di build basati su `Make`, aspetto studiato anche in letteratura prendendo in esame diversi progetti *open source* di larga scala, documentando più di un milione di dipendenze non dichiarate

nei sistemi di build. Molte di queste dipendenze non dichiarate sono riconosciute dagli sviluppatori e diversi di queste sono state arbitrariamente lasciate in modo da snellire il processo di build [BMA⁺17]. Questo problema dunque si manifesta anche nel contesto di questo progetto con la compilazione della libreria FMILib, la quale utilizza CMake come sistema di building.

La compilazione di FMILib è gestita tramite il plugin Gradle per interagire con CMake chiamato `gradle-cmake`³, che integra il ciclo di build CMake direttamente nel task graph di Gradle. Il modulo `fmilib` registra un target CMake che punta al `CMakeLists.txt` della libreria e aggiunge un task `cmakeInstall` che, dopo la compilazione, esegue `cmake --install` per ciascuna piattaforma rilevata nella directory di build, producendo gli artefatti installati in una struttura `fmilib-install/<piattaforma>` con le sottocartelle `include`, `lib` e `bin`. Questa struttura è quella referenziata dai moduli `fmu-kt` e `backend` per configurare le opzioni di cinterop e di linking. La figura 4.2 offre una visione generale del meccanismo di compilazione della libreria FMILib.

Il modulo `fmu-kt` dichiara una configurazione Gradle personalizzata (`cLibrary`) per esprimere la dipendenza sugli artefatti compilati di `fmilib`, e configura il cinterop di Kotlin/Native puntando all'header `fmilib.h` e alla directory degli include della piattaforma corrente. Poiché la piattaforma host è rilevata a runtime tramite `OperatingSystem.current()`, viene registrato un solo target nativo per build – quello corrispondente al sistema su cui si esegue Gradle – evitando la necessità di cross-compilazione. La stessa logica si applica al modulo `backend`.

Su Windows è presente una configurazione aggiuntiva: al termine del linking dell'eseguibile, un task di tipo `Copy` provvede a copiare la `.dll` di FMILib nella stessa directory dell'eseguibile prodotto, poiché su quella piattaforma il loader del sistema operativo richiede che le librerie condivise siano nella medesima directory del binario o nel `PATH`.

Pipeline CI/CD La pipeline di integrazione continua è definita tramite GitHub Actions e articolata in più workflow con scopi distinti. Il workflow principale (`CI-build.yml`), attivato sul branch `main`, esegue in parallelo tre job tramite una strategia a matrice: uno per Linux x64 su `ubuntu-latest`, uno per macOS ARM64

³GitHub contenente il codice del plugin `gradle-cmake`.

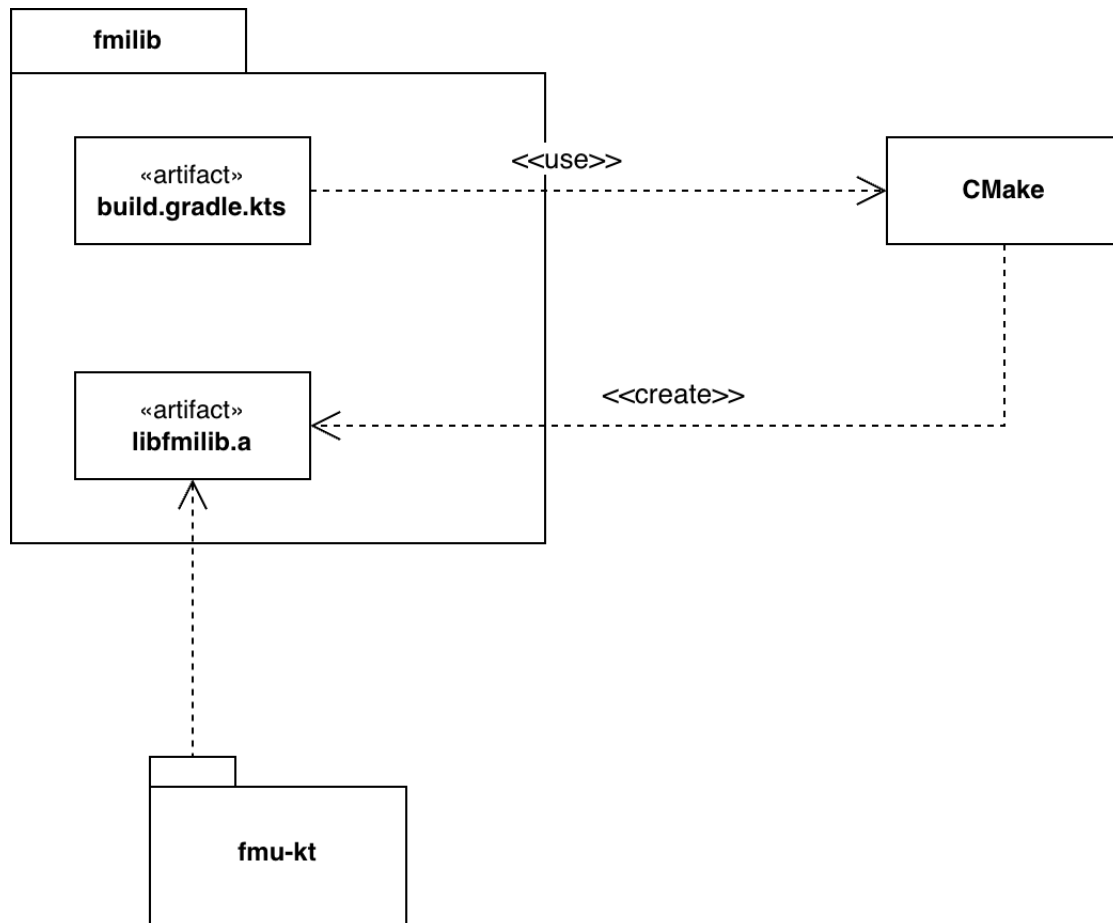


Figure 4.2: Diagramma rappresentante il processo di compilazione della libreria FMILib.

su `macos-latest` e uno per Windows x64 su `windows-latest`. Ciascun job esegue l'intero build Gradle – che include compilazione di FMILib, compilazione del codice Kotlin, ed esecuzione dei test – e al termine avvia il backend compilato effettuando una chiamata all'endpoint `/health` per verificare che il server risponda correttamente. In caso di fallimento dei test, i report vengono caricati come artefatti per facilitare la diagnosi. Gli artefatti di FMILib prodotti da ogni job vengono anch'essi caricati, rendendoli disponibili per eventuali download manuali.

Sono presenti workflow separati (`build-fmlib.yml` e `build-backend.yml`) attivati sui branch di sviluppo, con scopo di feedback rapido durante lo sviluppo: il primo isola la build e i test di FMILib e `fmu-kt`, il secondo si concentra sulla build e sul health check del backend. Questi workflow non vengono eseguiti sul branch `main`, dove è il workflow CI principale a coprire l'intero stack.

Containerizzazione con Docker Il progetto include un'immagine Docker che incapsula il backend compilato per Linux x64, pensata per facilitare il deployment del server in ambienti containerizzati senza dipendenze dall'ambiente host. Il workflow `docker-test.yml`, attivato sul branch `main`, si occupa di costruire l'immagine tramite Docker e di avviarla come container, verificando la disponibilità del servizio tramite polling sull'endpoint `/health`.

4.7 Implementazione backend

Il backend è implementato tramite il framework Ktor con un server Coroutine-based I/O (CIO), scelta guidata dalla necessità di produrre un eseguibile nativo autonomo senza dipendenza dalla JVM: tra i server engine disponibili in Ktor, CIO è l'unico compatibile anche con il target Kotlin/Native, in quanto gli altri engine si appoggiano a librerie che richiedono la JVM a tempo di esecuzione. La configurazione del server installa tre plugin: `ContentNegotiation`, che gestisce la serializzazione e deserializzazione automatica dei payload JSON tramite `kotlinx.serialization`; `Cross-Origin Resource Sharing (CORS)`, che abilita le richieste cross-origin necessarie nel caso il frontend venga servito da un'origine diversa; e `StatusPages`, che intercetta le eccezioni non gestite e le traduce in risposte HTTP con codice di errore appropriato, evitando che il server

esponga stack trace al client. Il routing è dichiarato in un blocco dedicato che mappa ciascun endpoint al metodo corrispondente di `RequestHandler`, mantenendo la definizione delle rotte separata dalla logica applicativa come descritto nel capitolo di design.

4.8 Implementazione frontend

Il frontend Web è implementato con Compose Multiplatform⁴, il framework di JetBrains che porta il modello di UI dichiarativa di Jetpack Compose al di fuori dell'ecosistema Android, consentendo di definire l'interfaccia utente in Kotlin anche tra piattaforme diverse. La navigazione tra le schermate è gestita tramite la libreria `Voyager`⁵, che integra il concetto di `Screen` direttamente nel modello a componenti di Compose: ciascuna schermata implementa l'interfaccia `Screen` e definisce il proprio contenuto nel metodo `Content()`, mentre il `Navigator` di Voyager gestisce lo stack di navigazione e le transizioni. Il punto di ingresso dell'applicazione istanzia il `Navigator` con `HomeScreen` come schermata iniziale.

La visualizzazione dei risultati di simulazione nella `ChartScreen` è realizzata tramite la libreria `KoalaPlot`⁶, che fornisce componenti Compose Multiplatform per la produzione di grafici. Per ciascuna variabile campionata viene prodotta una serie temporale, rappresentata come `LinePlot` con un colore distinto estratto da una palette predefinita. Gli assi del grafico vengono configurati dinamicamente in base ai valori effettivi dei dati, con un padding verticale proporzionale all'intervallo dei valori per evitare che le serie tocchino i bordi del grafico.

La selezione del file FMU da caricare è gestita tramite la libreria `FileKit`⁷, che astrae le API native di apertura file delle diverse piattaforme esponendo un'interfaccia Compose Multiplatform unificata. Questo consente alla `HomeScreen` di aprire il file picker nativo del sistema operativo e ottenere il file selezionato come `PlatformFile`, indipendentemente dalla piattaforma su cui il frontend è in esecuzione, per poi passarne il contenuto al modulo client per l'upload al backend.

⁴Pagina Web ufficiale di Compose Multiplatform.

⁵Documentazione della libreria `Voyager`.

⁶Pagina GitHub della libreria `KoalaPlot`.

⁷Documentazione della libreria `FileKit`

Chapter 5

Valutazione

La valutazione del sistema è stata condotta su due livelli complementari. Il primo riguarda la correttezza funzionale dei componenti: tramite test unitari e di integrazione si verifica che la logica di dominio, il backend e la loro interazione producano i risultati attesi in isolamento, indipendentemente dalla piattaforma su cui vengono eseguiti. Il secondo livello riguarda invece la correttezza multipiattaforma: la pipeline di CI esegue l'intera suite di test e una verifica end-to-end del backend su Linux x64, macOS ARM64 e Windows x64, garantendo che il comportamento del sistema sia consistente su tutte le piattaforme target.

Le sezioni seguenti descrivono nel dettaglio le strategie adottate per ciascuno dei due livelli, le scelte che hanno guidato la struttura dei test e i risultati ottenuti.

5.1 Test unitari e di integrazione

Strategia di testing e uso dei mock La strategia di testing adottata mira a verificare i componenti del sistema in isolamento, evitando dipendenze tra moduli che renderebbero i test fragili e difficili da diagnosticare. A tale scopo, il backend non istanzia mai direttamente `DefaultFmuService`, ma dipende dall'interfaccia `FmuService`, che viene sostituita nei test da `FakeFmuService`, ovvero un'implementazione fittizia con comportamento prevedibile e controllabile. Analogamente, `ResourceManager` viene sostituito da `FakeResourceManager`, che simula le operazioni sul filesystem senza accedervi realmente. Questa architettura permette di

testare l'intera logica del `RequestHandler` – dalla gestione degli errori alla serializzazione delle risposte – senza che i test dipendano dalla presenza di un FMU reale o di un filesystem configurato, rendendo la suite eseguibile in qualsiasi ambiente CI senza setup aggiuntivo.

Test del modulo `fmu-kt` I test del modulo `fmu-kt` sono organizzati in due livelli. Il primo comprende test unitari sui componenti specifici di macOS ARM64 – `DescriptionParser`, `FileManager`, `FmiHeaderSynthesiser` e `FmuRecompiler` – che verificano il comportamento dei singoli componenti della pipeline di ricompilazione in isolamento. Il secondo livello è costituito da una suite di test di integrazione che mette alla prova `FmuManager` su FMU reali, verificando il corretto funzionamento dell'intera catena: caricamento, estrazione dei metadati e simulazione.

Suite dinamica su FMU reali con Kotest La suite di integrazione è implementata tramite Kotest ed è progettata per essere estensibile in previsione di sviluppi futuri. I test vengono generati dinamicamente a partire da un sottomodulo Git che raccoglie una collezione di FMU di riferimento¹: per ciascun FMU presente nel sottomodulo viene prodotta una serie di test case relativi al ciclo di vita, all'estrazione dei metadati e alla simulazione che compaiono come voci distinte nel report.

L'esecuzione di ciascun test è condizionata da una funzione di guardia, `fmuGuard`, che filtra i modelli in base alla versione FMI supportata: nella configurazione attuale vengono abilitati solo gli FMU di versione 2.0, che è l'unica versione supportata dall'applicativo. La guardia è espressa come condizione booleana facilmente modificabile: nel momento in cui si volesse estendere il supporto a FMI 1.0 o 3.0, sarebbe sufficiente aggiornare la condizione di filtraggio per includere la nuova versione, e i test corrispondenti si attiverebbero automaticamente prima ancora di iniziare l'implementazione, abilitando un approccio test-driven allo sviluppo delle nuove funzionalità.

Su macOS ARM64 è presente un ulteriore livello di filtraggio: la guardia esclude gli FMU privi di sorgenti C, poiché la pipeline di ricompilazione descritta

¹GitHub contenente collezione di FMU utilizzati per il testing.

nella Sezione 4.5 richiede la disponibilità dei sorgenti per produrre il fat binary compatibile con Apple Silicon. Gli FMU precompilati senza sorgenti vengono quindi marcati come disabilitati con una motivazione esplicita nel report, mantenendo visibilità sul fatto che l'esclusione è intenzionale e non un errore.

Ogni test case gestisce autonomamente la creazione e la pulizia di una directory temporanea dedicata tramite un blocco `try/finally`, garantendo che lo stato del filesystem sia ripristinato dopo ogni esecuzione indipendentemente dall'esito del test.

5.2 Copertura multipiattaforma nella CI

L'esecuzione della suite di test è integrata direttamente nella pipeline di CI descritta nella Sezione 4.6: il task `build` di Gradle, invocato su ciascuno dei tre runner – `ubuntu-latest`, `macos-latest` e `windows-latest` – include l'esecuzione completa dei test come parte del ciclo di build ordinario. In questo modo ogni push sui branch principali produce una verifica automatica della correttezza del sistema su tutte le piattaforme target, senza richiedere passi separati.

Su Windows è necessaria una configurazione aggiuntiva: la directory contenente la `.dll` di `FMILib` viene aggiunta dinamicamente alla variabile d'ambiente `PATH` prima dell'esecuzione del task di test, poiché il loader di Windows richiede che le librerie condivise siano raggiungibili tramite quella variabile.

Al termine della build, il workflow verifica la correttezza end-to-end del backend avviando l'eseguibile prodotto ed effettuando una chiamata all'endpoint `/health`, confermando che il server sia in grado di rispondere correttamente dopo l'intero ciclo di compilazione e linking sulla piattaforma corrente.

Chapter 6

Conclusioni e Sviluppi Futuri

6.1 Limitazioni

Il sistema sviluppato presenta alcune limitazioni che restano aperte al termine del lavoro. La più significativa riguarda il supporto allo standard FMI: l'applicativo supporta esclusivamente la versione 2.0 in modalità Co-Simulation, escludendo sia la versione 1.0 che la più recente 3.0. Estendere il supporto richiederebbe uno studio approfondito delle API di FMILib per ciascuna versione, che differiscono non solo nei nomi dei simboli nativi – `fmi2_*` per la versione 2.0, `fmi3_*` per la 3.0 – ma anche nella struttura del lifecycle e nelle modalità di simulazione.

Una seconda limitazione riguarda la pipeline di ricompilazione degli FMU su macOS ARM64. La soluzione attuale si affida a header FMI 2.0 sintetizzati al momento della compilazione, in assenza di un meccanismo che garantisca l'utilizzo degli header ufficiali distribuiti dallo standard. Sebbene la sintesi produca header funzionalmente equivalenti per i casi trattati, un approccio più robusto prevederebbe l'inclusione degli header ufficiali all'interno del progetto, eliminando la dipendenza dalla fase di sintesi e rendendo la toolchain di ricompilazione più affidabile su FMU con sorgenti che fanno uso di funzionalità meno comuni dell'API FMI.

Una terza limitazione legata all'ecosistema e riguarda la disponibilità delle librerie standard di Kotlin nel target Kotlin/Native. Alcune API della libreria standard Kotlin, in particolare quelle relative alla navigazione e alla manipolazione

del filesystem, non sono disponibili o hanno supporto limitato nel target nativo, rendendo necessaria l'implementazione manuale di funzionalità che in altri contesti sarebbero coperte da librerie consolidate. Questo ha influenzato in particolare lo sviluppo dei componenti specifici per macOS ARM64, dove operazioni per l'interazioni con il filesystem che sarebbero state banali in un contesto JVM, hanno richiesto l'utilizzo diretto delle API POSIX tramite `cinterop`. Si tratta di una limitazione dell'ecosistema Kotlin/Native che tende a ridursi con le nuove versioni del compilatore, ma che al momento dello sviluppo ha rappresentato un vincolo concreto.

Infine, il sistema non dispone attualmente di funzionalità per l'esportazione dei risultati di simulazione. I dati prodotti sono visualizzabili tramite l'interfaccia grafica, ma non è possibile salvarli in un formato standard come CSV o JSON per un'analisi successiva con strumenti esterni.

6.2 Sviluppi futuri

Gli sviluppi futuri più naturali del progetto seguono direttamente le limitazioni descritte. L'estensione del supporto a FMI 3.0 e FMI 1.0 rappresenta la direzione più rilevante; l'architettura attuale offre una base favorevole per questo sviluppo: `FmuService` espone un'interfaccia agnostica alla versione FMI (`load`, `getInfo`, `simulate`), e sarebbe possibile introdurre una factory che, ricevuta la versione FMI rilevata in fase di caricamento, produca l'implementazione concreta di `FmuManager` appropriata. Ogni implementazione gestirebbe il proprio lifecycle e le proprie chiamate native dietro un'interfaccia comune che astrae le operazioni fondamentali, confinando le differenze semantiche tra le versioni nelle implementazioni specifiche. L'infrastruttura di test dinamica già predisposta consentirebbe inoltre di adottare un approccio test-driven: aggiornando la condizione di filtraggio della suite, i test sugli FMU della nuova versione si attiverebbero prima ancora di iniziare l'implementazione, mantenendo il ciclo di sviluppo guidato dai casi di test.

L'aggiunta di una funzione di esportazione dei risultati in formato CSV o JSON rappresenta invece uno sviluppo ad alto impatto pratico, che renderebbe il sistema più utile in contesti di ricerca e analisi dove i dati di simulazione devono essere

elaborati con strumenti esterni.

Un'ulteriore direzione di sviluppo riguarda il vincolo per cui il sistema gestisce un solo modello FMU alla volta. Questa scelta semplifica il modello del dominio e l'architettura, permettendo di focalizzarsi sullo sviluppo di un prototipo che permetta l'utilizzo esclusivo di Kotlin Multiplatform, ma esclude scenari in cui sia necessario co-simulare più componenti contemporaneamente. Nell'ecosistema degli strumenti *software* costruiti attorno lo standard FMI esistono soluzioni proprio pensate a questo scopo: FIDE[CLT⁺16], ad esempio, è un ambiente di sviluppo che permette di disporre graficamente una collezione di modelli FMU e di generare un eseguibile che ne co-simula l'insieme tramite un algoritmo dedicato. Estendere il sistema descritto in questa tesi in tale direzione richiederebbe l'introduzione di un componente che si occupi di orchestrare più **FmuManager** contemporaneamente, sincronizzando le simulazioni e gestendo il flusso di dati prodotto dai modelli — queste funzionalità tuttavia esula dagli obiettivi originari del progetto ma rappresenta un'interessante estensione naturale dell'architettura attuale.

6.3 Conclusioni

Il lavoro descritto in questa tesi dimostra che è possibile sviluppare un applicativo full stack che interagisce con librerie native utilizzando un unico linguaggio di programmazione. L'obiettivo posto all'inizio del progetto, ovvero costruire un sistema funzionante per la gestione e la simulazione di file FMU, dal binding con la libreria C fino all'interfaccia utente, senza mai uscire dall'ecosistema Kotlin, è stato raggiunto su tre piattaforme distinte: Linux x64, macOS ARM64 e Windows x64. Le sfide incontrate lungo il percorso, dalla configurazione del linker alla ricompilazione degli FMU su Apple Silicon, hanno richiesto soluzioni non banali, ma tutte affrontabili rimanendo all'interno del linguaggio scelto.

Kotlin Multiplatform si è rivelato uno strumento concretamente adatto a questo tipo di scenario. La possibilità di condividere la logica applicativa tra piattaforme diverse, producendo binari nativi autonomi e mantenendo al contempo un meccanismo diretto di interoperabilità con il codice C tramite **cinterop**, rappresenta una combinazione di caratteristiche che poche alternative offrono con lo stesso grado di integrazione. Per le aziende che operano in contesti dove le librerie native sono

una componente irrinunciabile, come quelle operanti in ingegneria e simulazione, Kotlin potrebbe rappresentare una scelta convincente: consente di unificare lo stack tecnologico senza sacrificare l'accesso al nativo, e la natura multiplatforma di KMP riduce la necessità di mantenere codebase separate per sistemi operativi diversi.

Alcune delle limitazioni incontrate durante lo sviluppo non sono intrinseche all'approccio, ma riflettono lo stato attuale di maturità dell'ecosistema Kotlin/Native. La disponibilità limitata di alcune API della libreria standard nel target nativo, la versione di glibc datata inclusa nel sysroot di Konan, e la necessità di configurare manualmente le opzioni di compilazione e linking sono aspetti che tendono a migliorare con ogni nuova versione del compilatore. Una maggiore adozione di Kotlin in ambito industriale – in particolare per scenari che richiedono interazione con il nativo – potrebbe accelerare questo processo, spingendo JetBrains a investire ulteriormente nel supporto a questi casi d'uso: dalla possibilità di selezionare la versione di glibc target, all'automatizzazione di parte della configurazione di compilazione e linking che oggi richiede una conoscenza approfondita di Gradle e della toolchain nativa.

Guardando avanti, l'evoluzione di Kotlin/Native lascia spazio a un ottimismo: le limitazioni incontrate durante lo sviluppo – dalla versione di glibc nel sysroot di Konan alla disponibilità parziale di alcune API della libreria standard nel target nativo – non sono strutturali, ma riflettono lo stato attuale di un ecosistema in rapida crescita. Una maggiore adozione di questo approccio in ambito industriale potrebbe accelerarne la maturazione, rendendo Kotlin una scelta sempre più solida per chi vuole coprire l'intero stack – dal codice nativo all'interfaccia utente – senza frammentare le competenze del team né moltiplicare la base di codice da mantenere.

List of Figures

2.1	Schema rappresentate le entità del modello del dominio.	15
3.1	Diagramma rappresentante relazione tra i moduli dell'applicativo. .	18
3.2	Diagramma delle dipendenze di <code>DefaultFmuService</code>	20
3.3	Design di <code>RequestHandler</code>	22
4.1	Struttura di <code>FmuRecompiler</code>	33
4.2	Diagramma rappresentante il processo di compilazione della libreria FMILib.	35

LIST OF FIGURES

Bibliography

- [AB⁺20] Marat Akhin, Mikhail Belyaev, et al. Kotlin language specification: Kotlin/Core. Technical report, JetBrains / JetBrains Research, 2020. Version 1.9-rfc+0.1.
- [BMA⁺17] Cor-Paul Bezemer, Shane McIntosh, Bram Adams, Daniel M. Germán, and Ahmed E. Hassan. An empirical study of unspecified dependencies in make-based build systems. *Empir. Softw. Eng.*, 22(6):3117–3148, 2017.
- [CLB⁺16] Fabio Cremona, Marten Lohstroh, David Broman, Marco Di Natale, Edward A. Lee, and Stavros Tripakis. Step revision in hybrid co-simulation with FMI. In *2016 ACM/IEEE International Conference on Formal Methods and Models for System Design, MEMOCODE 2016, Kanpur, India, November 18-20, 2016*, pages 173–183. IEEE, 2016.
- [CLT⁺16] Fabio Cremona, Marten Lohstroh, Stavros Tripakis, Christopher X. Brooks, and Edward A. Lee. FIDE: an FMI integrated development environment. In Sascha Ossowski, editor, *Proceedings of the 31st Annual ACM Symposium on Applied Computing, Pisa, Italy, April 4-8, 2016*, pages 1759–1766. ACM, 2016.
- [GTB⁺18] Cláudio Gomes, Casper Thule, David Broman, Peter Gorm Larsen, and Hans Vangheluwe. Co-simulation: A survey. *ACM Comput. Surv.*, 51(3):49:1–49:33, 2018.
- [MCK⁺24] Gunter Mussbacher, Benoît Combemale, Jörg Kienzle, Lola Burgueño, Antonio García-Domínguez, Jean-Marc Jézéquel, Gwendal Jouneaux,

- Djamel Eddine Khelladi, Sébastien Mosser, Corinne Pulgar, Houari A. Sahraoui, Maximilian Schiedermeier, and Tijs van der Storm. Polyglot software development: Wait, what? *IEEE Softw.*, 41(4):124–133, 2024.
- [MKL17] Philip Mayer, Michael Kirsch, and Minh Anh Le. On multi-language software development, cross-language links and accompanying tools: a survey of professional software developers. *J. Softw. Eng. Res. Dev.*, 5:1, 2017.
- [MM19] Bruno Gois Mateus and Matias Martinez. An empirical study on quality of android applications written in kotlin language. *Empir. Softw. Eng.*, 24(6):3356–3393, 2019.
- [MM20] Bruno Gois Mateus and Matias Martinez. On the adoption, usage and evolution of kotlin features in android development. In Maria Teresa Baldassarre, Filippo Lanubile, Marcos Kalinowski, and Federica Sarro, editors, *ESEM '20: ACM / IEEE International Symposium on Empirical Software Engineering and Measurement, Bari, Italy, October 5-7, 2020*, pages 15:1–15:12. ACM, 2020.
- [Mod14] Modelica Association. Functional mock-up interface for model exchange and co-simulation, July 2014. Version 2.0.
- [US18] Phillip Merlin Uesbeck and Andreas Stefik. A randomized controlled trial on the impact of polyglot programming in a database context. In Titus Barik, Joshua Sunshine, and Sarah E. Chasins, editors, *9th Workshop on Evaluation and Usability of Programming Languages and Tools, PLATEAU@SPLASH 2018, Boston, Massachusetts, USA, November 5, 2018*, volume 67 of *OASICS*, pages 1:1–1:8. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.
- [YLWC23] Haoran Yang, Weile Lian, Shaowei Wang, and Haipeng Cai. Demystifying issues, challenges, and solutions for multilingual software development. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*, pages 1840–1852. IEEE, 2023.